

Technical Guide: W: Web Pipe Device Y: Clipboard Device

Author: 13leader.net

Date: January 2026

Project Repo: <https://github.com/pjones1063/AspeQt-2k26>

1. Overview

W: Web Pipe Device (Thin Client Gateway)

The **W:** device transforms the Atari 8-bit into a "Thin Client" capable of interacting with modern web services. Instead of implementing complex network stacks (TCP/IP, SSL) on the Atari, the **W:** device acts as a transparent proxy. It forwards standard BASIC text commands to a backend script running on your PC via HTTP.

How It Works

The device operates using standard Atari I/O commands:

- **Write Mode (HTTP POST):** Opening **W:** for write allows you to buffer data. When you **CLOSE** the channel, AspeQt sends the buffered data as an HTTP POST request to the specified URL.
- **Read Mode (HTTP GET):** Opening **W:** for read immediately performs an HTTP GET request to the URL. Subsequent **INPUT** commands retrieve the server's response.

Running the Handler

To enable the **W:** and **Y:** devices, you must first load the handler into memory.

- **Manual Loading:** From the DOS menu, select option **L** (Binary Load) and type **HANDLER.XEX**.
- **Automatic Loading:** Rename **HANDLER.XEX** to **AUTORUN.SYS** on your boot disk. This ensures the network drivers load automatically every time you turn on the computer.

Code Example: The "Hello World" Loop

This example demonstrates sending data to a Python script and reading the response.

1. Atari BASIC (The Front-end)

Basic

```
10 DIM U$(40), MSG$(40), REPLY$(40)
20 U$="W:http://192.168.1.5:5000/atari"
30 MSG$="HELLO FROM 1979"

40 REM --- STEP 1: SEND DATA (POST) ---
50 OPEN #1,8,0,U$      : REM Open for Write
60 PRINT #1;MSG$       : REM Send the message
70 CLOSE #1           : REM Trigger the Network Request

80 REM --- STEP 2: READ RESPONSE (GET) ---
90 OPEN #1,4,0,U$      : REM Open for Read
100 INPUT #1,REPLY$    : REM Read text from PC
110 PRINT "SERVER SAID: ";REPLY$
120 CLOSE #1
```

2. Python/Flask (The Backend) *Requires Python installed on your host PC.*

Python

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/atari', methods=['GET', 'POST'])
def handle_atari():
    if request.method == 'POST':
        # Receive text from Atari
        data = request.data.decode('utf-8')
        print(f"Atari sent: {data}")
        return "OK"
    else:
        # Send text to Atari
        return "MESSAGE RECEIVED LOUD AND CLEAR"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Y: Clipboard Device (System Integration)

The **Y:** device provides a bi-directional bridge between the Atari 8-bit and your host computer's system clipboard (Windows, macOS, or Linux). This allows for rapid code transfer and data sharing without saving files to disk.

Usage Scenarios

- **Paste Code:** Copy BASIC listings from a website on your PC and paste them directly into the Atari editor.
- **Export Data:** Send program listings or variable data from the Atari to your PC's text editor.

Commands & Syntax

Action	Atari Command	Description
Write to PC	LIST "Y: "	Sends the current BASIC program to the PC clipboard.
Write to PC	PRINT #1;"TEXT"	Sends specific text to the PC clipboard.
Read from PC	ENTER "Y: "	Types the PC clipboard content into the Atari (like keystrokes).
Read from PC	COPY "Y: " "E: "	Displays PC clipboard content on the screen.

Code Example: Reading & Writing

Writing to the PC Clipboard

Basic

```
10 REM Send a variable to the PC Clipboard
20 A$="Copied from Atari"
30 OPEN #1,8,0,"Y:"
40 PRINT #1;A$
50 CLOSE #1
```

Reading from the PC Clipboard *Ensure you have copied text on your PC first.*

Basic

```
10 DIM BUFFER$(256)
20 OPEN #1,4,0,"Y:"
30 INPUT #1,BUFFER$
40 PRINT "I found this on your clipboard:"
50 PRINT BUFFER$
60 CLOSE #1
```

AspeQt-2k26: Application Examples Library for W:

This library contains a collection of "Hybrid Computing" examples that demonstrate the power of the **W: Device**. These examples offload heavy processing (AI, Web Scraping, SSL, Speech Synthesis) to a modern PC (Host) while using the Atari 8-bit as the frontend terminal.

General Prerequisites

For all examples below, you must have:

1. **Python 3.x** installed on your Host PC (Linux/Windows/macOS).
2. **Flask** installed (`pip install flask`).
3. The **PC's IP Address** configured in the BASIC .LST files (look for `U$="W:http://..."`).

1. Gemini AI Chatbot

Description: Turns the Atari into a client for Google's Gemini Large Language Model. You can type natural language questions ("Who is Captain Kirk?") and receive intelligent, summarized answers on the Atari screen.

- **BASIC File:** `GEMINI_AI.LST`
- **Python Script:** `gemini_ai.py`
- **Dependencies:** `pip install google-generativeai`
- **Setup:** You must edit `gemini_ai.py` and insert your Google API Key.
- **Usage:** Run the Python script. Run the BASIC program. Type a prompt and wait for the AI response.

2. Basic Graphical Weather Station

Description: Fetches live weather data from the Open-Meteo API for a specific location (configured in Python) and displays it on the Atari using **GRAPHICS 5** (4-color mode). It draws custom pixel-art icons for Sun, Clouds, Rain, or Snow.

- **BASIC File:** `WEATHER.LST`
- **Python Script:** `weather.py`
- **Dependencies:** `pip install requests`
- **Usage:**
 1. Edit `weather.py` to set your Latitude/Longitude (default is Lincoln, ON).

2. Run the server.
3. Run the BASIC code to see the graphical dashboard.

3. Cloud Notepad (Persistence Demo)

Description: Demonstrates Reading and Writing to the PC's hard drive. You can type a note on the Atari, save it, turn the Atari off, and retrieve the note later. It acts as a "Cloud Drive" for text.

- **BASIC File:** `CLOUD_NOTE.LST`
- **Python Script:** `cloud_note.py`
- **Dependencies:** `flask` (Standard)
- **Usage:**
 - Select **1** to Read the current note.
 - Select **2** to Write/Overwrite the note.
 - The note is saved as `atari_notes.txt` on the PC.

4. Text-to-Speech Synthesizer

Description: The "Talking Atari." Any text typed on the Atari is sent to the PC, where it is read aloud using the Linux `espeak` engine. This demonstrates zero-latency media triggering.

- **BASIC File:** `TTS_SERVER.LST`
- **Python Script:** `tts_server.py`
- **Dependencies:**
 - Python: `flask`
 - Linux System: `sudo apt install espeak`
- **Usage:** Type a phrase on the Atari (e.g., "Hello World"). The PC speakers will speak the phrase immediately.

5. Web News Scraper

Description: Connects to a modern news website (like CBC or BBC), scrapes the HTML, strips out ads/javascript, and delivers a plain-text headline summary to the Atari.

- **BASIC File:** `SCRAPER.LST`
- **Python Script:** `scraper.py`

- **Dependencies:** `pip install requests beautifulsoup4`
- **Usage:** Run the program to get a scrolling list of the top 5 news headlines.

6. Linux Remote Execution

Description: Allows the Atari to execute shell commands on the Linux Host. It includes a safety whitelist to prevent unauthorized access.

- **BASIC File:** `EXEC_LINUX.LST`
- **Python Script:** `exec_linux.py`
- **Dependencies:** `flask` (Uses standard `subprocess` library)
- **Usage:** The BASIC program sends a pre-defined command (like `ls -la` or `whoami`). The PC executes it and returns the terminal output to the Atari.

7. Windows Remote Control

Description: Similar to the Linux version but tailored for Windows Hosts. It can launch GUI applications (Calculator, Notepad) or open URLs in the default browser.

- **BASIC File:** `EXEC_WIN.LST`
- **Python Script:** `exec_win.py`
- **Dependencies:** `flask` (Uses standard `subprocess` library)
- **Usage:** Select an app from the menu (e.g., "1. Calculator"). The app will launch on the Windows PC screen.

8. File Upload / Telemetry

Description: A utility to upload raw data or text files from the Atari to the PC. This can be used for saving game high scores, sensor logs, or telemetry data.

- **BASIC File:** `FILE_UPLOAD.LST`
 - **Python Script:** `file_upload.py`
 - **Dependencies:** `flask`
 - **Usage:** The BASIC program simulates a file upload. The data is saved to a timestamped file in the `uploads/` folder on the PC.
-

Quick Dependency Install Command

To run all examples in this library, install the following packages on your Host PC:

Linux/Mac:

Bash

```
sudo apt install espeak
```

```
pip install flask requests beautifulsoup4 google-generativeai psutil
```

Windows:

PowerShell

```
pip install flask requests beautifulsoup4 google-generativeai psutil
```